

DOCUMENTAZIONE DI PROGETTO

Team D13
TASK R1

1. ORGANIZZAZIONE

1.1 STRUTTURA DEL TEAM

Sabino Gabriele Breglia	s.breglia@studenti.unina.it	DE9000097
Marcello Esposito	marcello.esposito6@studenti.unina.it	M63001768

URL della repository di partenza:

https://github.com/Testing-Game-SAD-2023/A13/tree/D13_TaskR1_UploadRobot

URL della repository del progetto svolto:

https://github.com/sabinogb/R1-task/tree/D13_TaskR1R2_UploadRobot_CaricamentoSuggerimenti_MigrazioneMongoDB

1.2 DEFINIZIONE DEGLI OBIETTIVI DI PROGETTO

Task assegnato

Il task da svolgere consiste nel **migliorare la robustezza della funzionalità di upload** delle classi e dei relativi unit test. In particolare, i problemi principali che sono stati evidenziati all'assegnazione del task sono due:

1. in caso di upload fallito, sul volume condiviso possono persistere dei file da scartare,
2. in caso di upload fallito, all'amministratore non viene comunicato alcun messaggio di errore che indica in maniera dettagliata la causa dell'errore.

Approccio di sviluppo seguito

Sono stati seguiti i principi **Agile**, con particolare riferimento all'**XP** che si adatta molto bene alla natura di questo task, essendo composto da numerosi piccoli subtask.

Più nel dettaglio, tra i principi cardine seguiti, si riportano:

- **Pair Programming:** è vantaggioso, dal momento che il team composto da due studenti, e permette inoltre di ridurre il tempo di validazione delle nuove modifiche, dato che queste vengono validate da entrambi in tempo reale,
- **Small Increments:** data la natura del task, il modo più semplice per effettuare modifiche e poter tornare ad uno stato consistente del sistema è stato quello di effettuare tanti piccoli incrementi (commit),

- **Testing:** per quanto non si possa parlare di TDD dato che non sono stati formalizzati i test né tantomeno automatizzati, le nuove modifiche nel sistema sono state continuamente testate per garantire il corretto funzionamento ad ogni nuovo incremento, o comunque un insieme piccolo di incrementi,
- **Simple Design:** si è cercato di mantenere le componenti quanto più semplici possibili,
- **Continuous Refactoring:** di volta in volta è stato continuamente valutato il caso di rifattorizzare un metodo o un servizio, anche grazie all'utilizzo di strumenti come **SonarQube**,
- **Timeboxing:** non sempre la valutazione dei tempi è stata precisa, dovendo rimandare alcune modifiche alla iterazione successiva.

Gestione del progetto

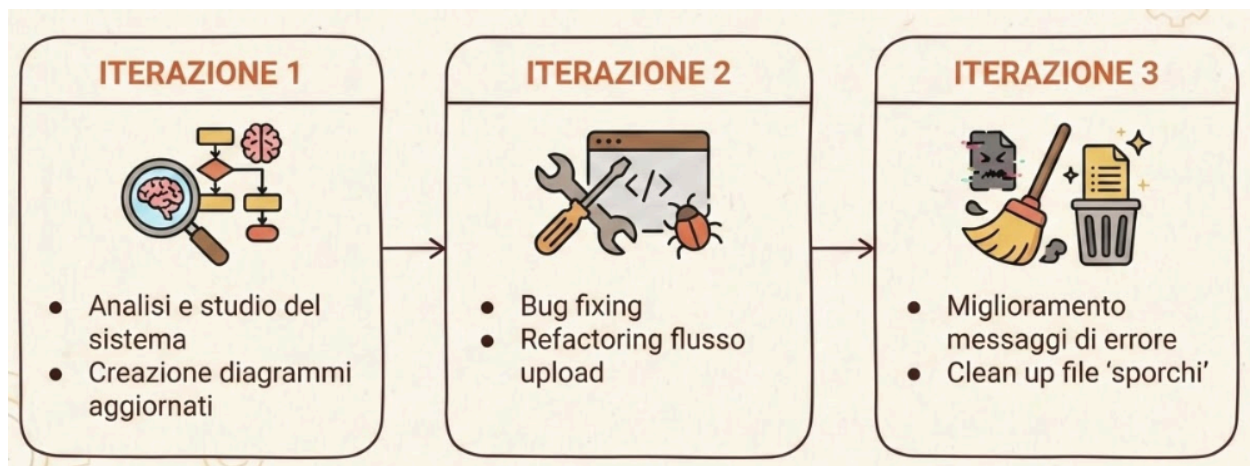
Seguito il modello di sviluppo del **framework SCRUM** data la natura di questo esame.

Nella prima iterazione è stato principalmente analizzato il sistema per definire il product backlog complessivo.

Successivamente, per ciascuna iterazione è stato deciso il **tema** per lo Sprint backlog, così da lavorare su requisiti della stessa tipologia.

I vari subtask, infatti, sono stati raggruppati per tipologia (refactoring, fix, documentazione, nuove funzionalità).

Il processo di sviluppo ha poi avuto anche diverse analogie con l'**UP**: è stata effettuata una prima fase di analisi e stima dei costi, per poi passare allo sviluppo dei requisiti Core (**bug fixing** e **refactoring**), seguiti dall'implementazione delle funzionalità rimanenti (es. error response più parlante) ed il rilascio del sistema tramite pull request.



2. ANALISI DEI REQUISITI PER IL TASK ASSEGNATO

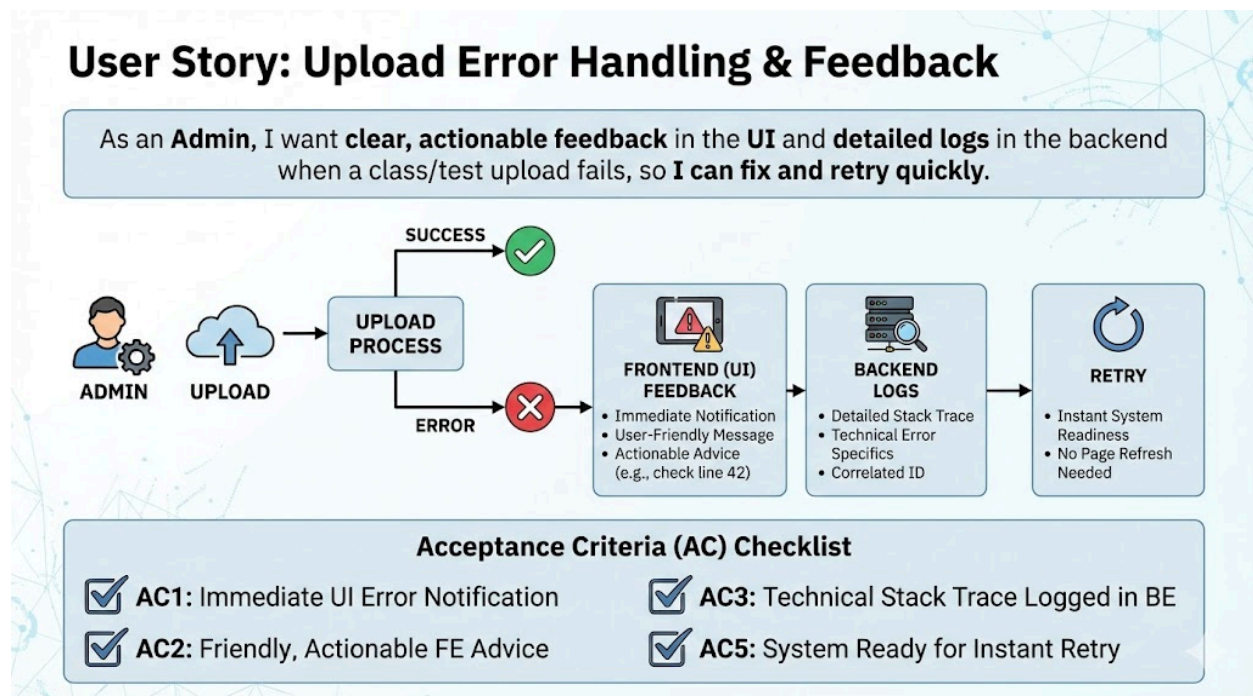
Analisi dello stato attuale del sistema

L'analisi svolta è da considerarsi in termini di qualità del software nel flusso di Upload, da cui sono stati evidenziati i seguenti problemi (anche con l'ausilio del tool **sonarQube**):

- La presenza di **code smell** e **antipattern**, che rendevano la gestione, la comprensione e l'aggiornamento del codice sorgente più complessi, richiedendo un importante lavoro di **refactoring**,
- **Bug** che impedivano il corretto upload della classe e dei test dei robot,
- **Documentazione** praticamente inesistente.

L'attività di **refactoring**, che ha accompagnato tutte e tre le iterazioni, è stata di fondamentale importanza soprattutto nella prima iterazione.

Il **refactoring**, a sua volta, ha portato alla luce alcuni **bug** che impedivano il corretto funzionamento dell'applicativo durante la fase di upload delle classi. Tali errori coinvolgono, oltre al servizio **T1**, anche i servizi **T7** e **T8**.



Dalla user story emergono i seguenti requisiti:

- Mostrare un **errore esplicativo** al FE seguendo il pattern **WHAT + WHY + HOW**,
- Uniformare i **log** e renderli più chiari,
- **Clean-up file** che persistono nel sistema anche in caso di errore durante l'upload, impedendo un nuovo upload per tale classe,
- Rendere **T7** e **T8** più parlanti sia nei log che nelle risposte inviate a **T1**.

Analisi dell'impatto

E' stato modificato principalmente il micro servizio **T1**. In particolare, i service **OpponentService** e **UploadOpponentService** e le rispettive **Util class** sono stati rifattorizzati e scomposti.

Inoltre è stata introdotta una modalità di debugging nel micro servizio **T7** per far persistere i file temporanei su cui genera la coverage per debugging, impostando un'apposita variabile d'ambiente.

3. PROGETTAZIONE DELLA SOLUZIONE

3.1 SUBTASK PRINCIPALI

- **Risoluzione bug - Generazione test Evosuite**

Nel caso in cui venga caricato in input un file zip **non contenente alcun file nella cartella TestReport/** nei test di Evosuite, questo comporterà un errore nella fase di upload.

Questo avviene perché in questo caso, non viene creato correttamente l'intero **albero di directory** nel volume condiviso **T0**, di conseguenza questo porta a far sì che una volta ricevuto il file di Coverage dal micro servizio **T8**, il metodo utilizzato per salvare il file non trovasse la parent directory corretta lanciando un'eccezione.

```
public static void writeStringToFile(String content, File file) throws IOException {
    try (FileWriter writer = new FileWriter(file)) {
        writer.write(content);
    }
}
```

Per risolvere questo problema è stato aggiornato il metodo di salvataggio del file, che ora **controlla prima se le parent directory** sono presenti ed in caso le crea iterativamente.

```
public void writeStringToFile(String content, File file) throws IOException {
    File parent = file.getParentFile();
    if (parent != null) {
        Files.createDirectories(parent.toPath());
    }
    try (FileWriter writer = new FileWriter(file)) {
        writer.write(content);
    }
}
```

- **Risoluzione bug - Duplici chiamate a Evosuite e Jacoco**

In seguito all'upload di una classe e del relativo archivio contenenti i test, se i file

contenenti le metriche di uno dei due robot fossero assenti, **T1 invocherebbe sia T7 che T8** per ricalcolare i file di coverage per entrambi i robot, invece che invocare esclusivamente la generazione del robot da cui sono stati generati i file di test.

```
if (!coverageFound[1]) {
    Path tmpFolder_ToZip = Paths.get(String.format("%s/%s/tmp_zip", volumeBasePath, classUTName));

    Files.createDirectories(Paths.get(String.format("%s/%s", tmpFolder_ToZip, Paths.get(BASE_SRC_PATH)));
    FileOperationUtil.copyDirectoryRecursively(toSrcPath, Paths.get(String.format("%s/%s", tmpFolder_ToZip, BASE_SRC_PATH)));

    Files.createDirectories(Paths.get(String.format("%s/%s", tmpFolder_ToZip, Paths.get(BASE_TEST_PATH)));
    FileOperationUtil.copyDirectoryRecursively(toTestPath, Paths.get(String.format("%s/%s", tmpFolder_ToZip, BASE_TEST_PATH)));

    FileOperationUtil.zipDirectory(String.format("%s/src", tmpFolder_ToZip), String.format("%s/src.zip", tmpFolder_ToZip));
    File zip = new File(String.format("%s/src.zip", tmpFolder_ToZip));

    String srcPackage = "";
    if (srcPackageNameSplit != null)
        srcPackage = String.join(".", srcPackageNameSplit) + ".";

    if (!zip.exists()) {
        System.err.println("Errore: Il file ZIP non è stato creato correttamente.");
        FileOperationUtil.deleteDirectoryRecursively(tmpFolder_ToZip);
    } else {
        EvoSuiteCoverageDTO coverageDTO = apiGatewayClient.callGenerateMissingEvoSuiteCoverage(classUTName, srcPackage, zip);
        FileOperationUtil.writeStringToFile(coverageDTO.getResultFjsszContent(), new File(String.format("%s/%s/coverage_fjssz.json", tmpFolder_ToZip, classUTName)));
    }

    Files.delete(zip.toPath());
    FileOperationUtil.deleteDirectoryRecursively(tmpFolder_ToZip);
}

if (!coverageFound[0]) {
    Path tmpFolder_ToZip = Paths.get(String.format("%s/%s/tmp_zip", volumeBasePath, classUTName));

    Files.createDirectories(Paths.get(String.format("%s/%s", tmpFolder_ToZip, Paths.get(BASE_SRC_PATH)));
    FileOperationUtil.copyDirectoryRecursively(toSrcPath, Paths.get(String.format("%s/%s", tmpFolder_ToZip, BASE_SRC_PATH)));

    Files.createDirectories(Paths.get(String.format("%s/%s", tmpFolder_ToZip, Paths.get(BASE_TEST_PATH)));
    FileOperationUtil.copyDirectoryRecursively(toTestPath, Paths.get(String.format("%s/%s", tmpFolder_ToZip, BASE_TEST_PATH)));

    FileOperationUtil.zipDirectory(String.format("%s/src", tmpFolder_ToZip), String.format("%s/src.zip", tmpFolder_ToZip));
    File zip = new File(String.format("%s/src.zip", tmpFolder_ToZip));

    if (!zip.exists()) {
        logger.error("Errore: Il file ZIP non è stato creato correttamente.");
        FileOperationUtil.deleteDirectoryRecursively(tmpFolder_ToZip);
    } else {
        JacocoCoverageDTO coverageDTO = apiGatewayClient.callGenerateMissingJacocoCoverage(zip);
        FileOperationUtil.writeStringToFile(coverageDTO.getResultFjacocoContent(), new File(String.format("%s/%s/coverage_fjacoco.json", tmpFolder_ToZip, classUTName)));
    }

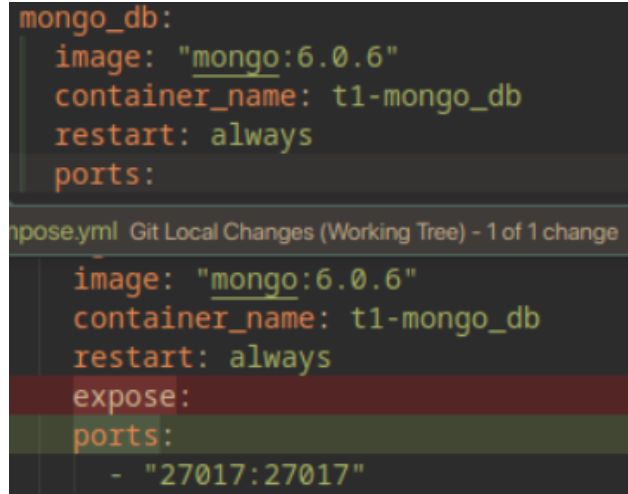
    Files.delete(zip.toPath());
    FileOperationUtil.deleteDirectoryRecursively(tmpFolder_ToZip);
}
```

Per risolvere questo problema, viene adesso controllato il **robotType** prima di invocare la coverage generation di EvoSuite o Jacoco, gestendo anche **robotType** non conosciuti.

```
private void generateCoverageIfNeeded(String robotType, boolean[] coverageFound,
    String classUTName, String[] srcPackageNameSplit,
    OpponentPaths opponentPaths) throws IOException {
    if (ROBOT_TYPE_EVOSUITE.equalsIgnoreCase(robotType)) {
        logger.info("Generating EvoSuite coverage for robot type {}", robotType);
        ensureEvoCoverageIfMissing(coverageFound[1], classUTName, srcPackageNameSplit,
            opponentPaths.srcPath, opponentPaths.testPath, opponentPaths.coveragePath);
    } else if (ROBOT_TYPE_RANDOOOP.equalsIgnoreCase(robotType)) {
        logger.info("Skipping EvoSuite coverage generation for robot type {}", robotType);
        ensureJacocoCoverageIfMissing(coverageFound[0], classUTName,
            opponentPaths.srcPath, opponentPaths.testPath, opponentPaths.coveragePath);
    } else {
        logger.error("Unknown robot type {}, skipping coverage generation", robotType);
    }
}
```

- **Risoluzione bug - Deploy (aggiornamento DockerFile)**

In seguito a un recente aggiornamento di Docker, i **DockerFile** di T1 e T5 presentano problemi di retrocompatibilità con la keyword **expose**, impedendo il deploy dei micro servizi, che è stata adesso aggiornata con **ports**.



```
mongo_db:
  image: "mongo:6.0.6"
  container_name: t1-mongo_db
  restart: always
  expose:
  ports:

compose.yml  Git Local Changes (Working Tree) - 1 of 1 change
  image: "mongo:6.0.6"
  container_name: t1-mongo_db
  restart: always
  expose:
  ports:
    - "27017:27017"
```

- **Risoluzione bug - Nome della classe nel form non corrispondente al nome classe nel file Java**

Per questa problematica sono state riscontrate due casistiche:

- **File di coverage non presenti**

Nel caso in cui non siano presenti i file di coverage, quando viene inviata la richiesta a **T7** e **T8** viene inviato anche il nome della classe, ma quando proveranno a generare i test per tale classe non riescono a trovarlo, causando un'eccezione nel flusso di Upload

- **File di coverage presenti**

Nel caso in cui siano presenti i file di coverage, l'upload andrà a buon fine, il problema è che nella fase di gioco quando si proverà a confrontare i propri risultati con i robot, mvn andrà in errore non trovando la classe con tale nome che sarà poi anche mostrato nella console della partita

Per risolvere questo problema è stato scelto di utilizzare l'approccio che impattasse di meno anche negli altri microservizi, andando ad introdurre un **Input Validation sul nome della classe**, verificando che il nome inserito nel form corrisponda al nome nel file Java prima di procedere con l'upload


```
String classNameFromSourceFile = JavaMetadataExtractor.getClassNameFromJavaSourceFile(classFileBytes);

if(classNameFromSourceFile == null) {
    throw new ClassValidationException(
        "error.validation.class.noDeclaration"
    );
}

if(!classNameFromSourceFile.equalsIgnoreCase(classUTName)) {
    throw new ClassValidationException(
        "error.validation.class.nameMismatch",
        classNameFromSourceFile, classUTName
    );
}
}
```

Errore caricamento classe

Mancata corrispondenza tra nome classe nel file e nome inserito nel form: • Nome nel file .java: 'FTPFile' • Nome inserito nel form: 'Calcolatrice' Correggere il campo 'Class Name' nel form in modo che corrisponda esattamente al nome della classe dichiarato nel file .java (verificare maiuscole/minuscole).

OK

• Uniformazione log con SLF4J

Il sistema di **log** interni in alcuni punti risulta non uniformato con chiamate che redirezionano output testuale verso lo standard output (usando **System.out.println**), adesso standardizzati con **SLF4J**.

```
public ResponseEntity<FileUploadResponse>
uploadOpponent(...) {
    ...
    // None del file e dimensione
    String classUTFileName = StringUtils.cleanPath(...);
    long size = classUTFile.getSize();
    System.out.println("Salvataggio di " +
classUTFileName + " nel filesystem condiviso");
}
```

```
public void saveClassUTFile(String classUTFileName,
String classUTName, MultipartFile classUTFile)
{
    logger.info("Salvataggio di {} nel filesystem
condiviso", classUTFileName);
}
```

• Modalità debug T7

Introdotta una modalità di debug in **T7** attivabile tramite apposita variabile d'ambiente, che permette di **preservare i file temporanei** che vengono elaborati per generare il file di coverage.

```

name: t7

<> Run All Services
services:
  <> Run Service
  app:
    image: mick0974/a13:t7-g31
    container_name: t7-controller
    restart: always
    environment:
      OTEL_EXPORTER_OTLP_ENDPOINT: "http://otel-
      OTEL_EXPORTER_OTLP_PROTOCOL: grpc
      DEBUG_PRESERVE_TMP: true

```

```

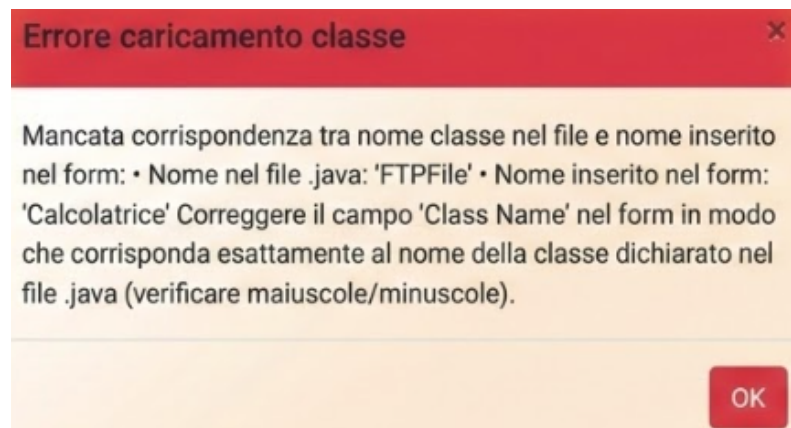
boolean preserveTmp = "true".equalsIgnoreCase(System.getenv("DEBUG_PRESERVE_TMP"));
if (preserveTmp) {
    logger.warn("[Compilation Service] DEBUG_PRESERVE_TMP=true, preserving compiler working dir: {}", config.getPathCompiler());
} else {
    FileUtils.deleteDirectoryRecursively(Paths.get(config.getPathCompiler()));
}

```

- **Errori più parlanti**

- **Nuovo modal per mostrare errori dettagliati al front-end**

In caso di errore in fase di upload, all'amministratore viene mostrato nel browser un pop up personalizzato che indica le cause dell'errore.



- **Global Exception Handler**

Seguendo quanto fatto nel micro servizio **T4**, è stato introdotto una nuova classe `GlobalExceptionHandler` che intercetta tutti gli errori nel micro servizio, per uniformare la risposta ritornata attraverso la nuova **`ErrorResponseDTO`** e **localizzare** i messaggi in base alla lingua dell'utente, prendendo tali messaggi dagli appositi file **`.properties`**.

```

/**
 * Global exception handler for REST controllers.
 * Provides uniform error responses across the application.
 */
@RestControllerAdvice
public class GlobalExceptionHandler {

```



```

/**
 * Handle file upload exceptions (file empty, invalid format, etc.)
 */
@ExceptionHandler(FileUploadException.class)
public ResponseEntity<ErrorResponseDTO> handleFileUploadException(FileUploadException ex, WebRequest request, Locale locale) {
    // Log the detailed error message
    logger.error("File upload error: {}", ex.getMessage(), ex.getCause());

    // Get localized message using message key if available, otherwise use detailed message
    String localizedMessage = getLocalizedMessage(ex, locale);

    ErrorResponseDTO error = new ErrorResponseDTO(
        HttpStatus.BAD_REQUEST.value(),
        "File Upload Error",
        localizedMessage
    );
    error.setPath(request.getDescription(false).replace("uri=", ""));
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
}

```

```

@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class ErrorResponseDTO {
    private LocalDateTime timestamp;
    private int status;
    private String error;
    private String message;
    private String path;

    public ErrorResponseDTO(int status, String error, String message) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = error;
        this.message = message;
    }

    public ErrorResponseDTO(int status, String message) {
        this.timestamp = LocalDateTime.now();
        this.status = status;
        this.error = getErrorName(status);
        this.message = message;
    }

    private String getErrorName(int status) {
        return switch (status) {
            case 400 -> "Bad Request";
            case 404 -> "Not Found";
            case 500 -> "Internal Server Error";
            default -> "Error";
        };
    }
}

```

- **Miglioramento log**
I messaggi di log interni sono stati resi più dettagliati.
- **Clean-up file in caso di errore**
Al fine di implementare una soluzione per il problema dei **file residui sul volume docker** da ripulire, è stata necessaria una **review del codice sorgente** del modulo **T1**, al fine di **ripercorrere tutte le operazioni su filesystem** che

vengono svolte a ogni tentativo di upload di una classe (creazione di cartelle, estrazione di archivi zip, copia di file).

Questo perché è stato introdotto un metodo di **rollback** che tenta di eliminare tutti i possibili file e dati creati in caso di errore.

```
try {
    // Save class file to filesystem
    classUTUploadService.saveClassUTFile(classUTFileName, classUTName, classUTFile);

    // Process and save opponents from ZIP
    uploadOpponentService.saveOpponentsFromZip(classUTFileName, classUTName, classUTFile, robotTestsZip);

    // Persist class metadata to database
    classUTUploadService.persistClassUTMetadata(classe, classUTFileName);

    // Success - build response
    FileUploadResponse response = new FileUploadResponse();
    response.setFileName(classUTFileName);
    response.setSize(classUTFile.getSize());
    response.setDownloadUri("/downloadFile");

    logger.info("Operazione completata con successo (uploadClassAndOpponents) - class: {}", classUTName);
    return ResponseEntity.ok(response);
} catch (Exception e) {
    // Rollback everything - just try to delete everything that might exist
    logger.error("Error during upload of {}, initiating rollback. Error: {}", classUTName, e.getMessage());
    performRollback(classUTName);
}
```

```
private void performRollback(String classUTName) {
    logger.warn("Starting rollback for class: {}", classUTName);

    // Try to delete metadata from database
    try {
        classUTUploadService.rollbackClassUTMetadata(classUTName);
    } catch (Exception e) {
        logger.debug("Rollback: metadata cleanup - {}", e.getMessage());
    }

    // Try to delete opponent data from database
    try {
        opponentRepository.deleteByClassUT(classUTName);
    } catch (Exception e) {
        logger.debug("Rollback: opponent DB cleanup - {}", e.getMessage());
    }

    // Try to notify external service to delete opponents
    try {
        apiGatewayClient.callDeleteAllClassUTOpponents(classUTName);
    } catch (Exception e) {
        logger.debug("Rollback: external service cleanup - {}", e.getMessage());
    }

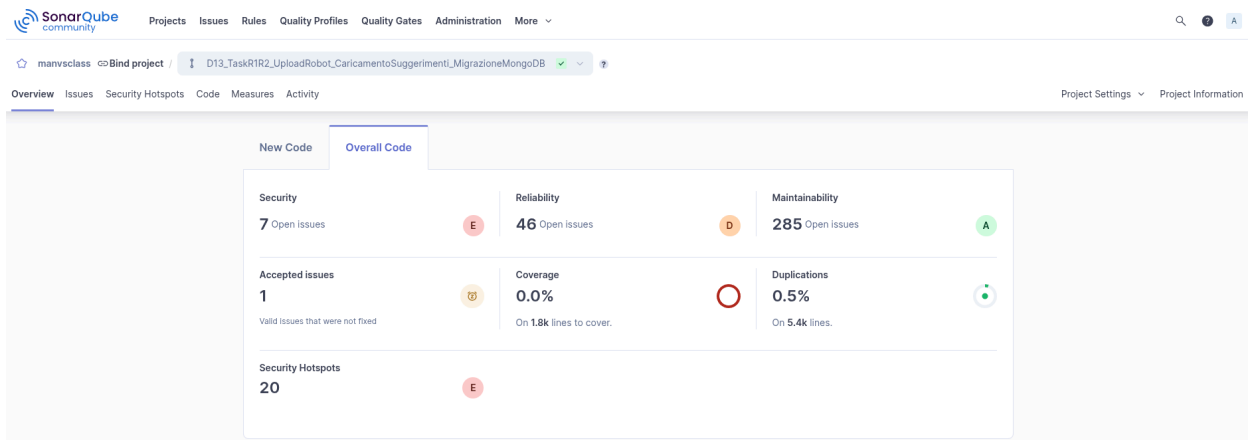
    // Try to delete all opponent files
    try {
        uploadOpponentService.rollbackOpponentFiles(classUTName);
    } catch (Exception e) {
        logger.debug("Rollback: opponent files cleanup - {}", e.getMessage());
    }

    // Try to delete class file
    try {
        classUTUploadService.rollbackClassUTFile(classUTName);
    } catch (Exception e) {
        logger.debug("Rollback: class file cleanup - {}", e.getMessage());
    }

    logger.info("Rollback completed for class: {}", classUTName);
}
```

- **Refactoring codice (con analisi SonarQube)**

Il codice sorgente presentava numerosi punti contrassegnati da **SonarQube** come code smells nel flusso di upload, che sono stati completamente rimossi nella seconda iterazione.



- **Nuovi diagrammi utili**

Sono stati generati i **class diagram** e i **sequence diagram** del progetto, sia prima dell'inizio del task, e in seguito alla terza iterazione.

Inoltre sono stati realizzati degli ulteriori diagrammi che spiegano il flusso di elaborazione dell'input e lo stato del file system step dopo step (mostrati nei punti successivi)

- **Validazione del form**

Il form per l'upload delle classi viene validato sia lato front-end che lato back-end. Viene introdotta una nuova classe, **FormValidation**, che contiene delle costanti che indicano la lunghezza massima di ciascun campo input testuale nella pagina HTML. Tali costanti vengono passate tramite **addObjects()** all'oggetto **ModelAndView** restituito all'utente, e inserite automaticamente all'interno degli attributi dei rispettivi elementi HTML.

```
@GetMapping("/upload") sabinogb +1
public ModelAndView showUploadRobots() {
    ModelAndView modelAndView = new ModelAndView( "opponents/opponents_upload");
    modelAndView.addObject( attributeName: "CLASS_NAME_MAX_LENGTH", FormValidation.CLASS_NAME_MAX_LENGTH);
    modelAndView.addObject( attributeName: "CLASS_CATEGORY_MAX_LENGTH", FormValidation.CLASS_CATEGORY_MAX_LENGTH);
    modelAndView.addObject( attributeName: "CLASS_DESCRIPTION_MAX_LENGTH", FormValidation.CLASS_DESCRIPTION_MAX_LENGTH);
    return modelAndView;
}
```

```

public class FormValidation { 6 usages sabinogb
    public static final int CLASS_NAME_MAX_LENGTH = 30; 2 usages
    public static final int CLASS_CATEGORY_MAX_LENGTH = 30; 2 usages
    public static final int CLASS_DESCRIPTION_MAX_LENGTH = 150; 2 usages
    public static final String[] DIFFICULTY_LEVELS = {"Beginner", "Intermediate", "Advanced"}; 1

    /*
     * Questo metodo restituisce true se i dati contenuti nell'oggetto ClassUT
     * rispettano la validazione.
     */
    public static boolean validateClassUT(ClassUT classe) throws NullPointerException { 1 usage
        Objects.requireNonNull(classe);
        Objects.requireNonNull(classe.getName());
        Objects.requireNonNull(classe.getDifficulty());

        //Il nome della classe deve essere presente.
        if(classe.getName().length() == 0 || classe.getName().length() > CLASS_NAME_MAX_LENGTH) {
            return false;
        }

        //La stringa che rappresenta la difficoltà deve essere una delle 3 possibili.
        boolean foundDifficulty = false;

        for(String difficulty : DIFFICULTY_LEVELS) {
            if(difficulty.equals(classe.getDifficulty())) {
                foundDifficulty = true;
                break;
            }
        }

        if(foundDifficulty == false) {
            return false;
        }

        if(classe.getCategory() != null) {
            if(classe.getCategory().size() > 3) {
                return false;
            }
            else {
                for(String category : classe.getCategory()) {
                    if(category.length() > CLASS_CATEGORY_MAX_LENGTH) {
                        return false;
                    }
                }
            }
        }

        if(classe.getDescription() != null) {
            if(classe.getDescription().length() > CLASS_DESCRIPTION_MAX_LENGTH) {
                return false;
            }
        }

        return true;
    }
}

```

```

if(FormValidation.validateClassUT(classe) == false)
{
    throw new ClassValidationException(
        "error.validation.class.failed"
    );
}

```

In particolare l'elenco dei controlli è:

INPUT	SIDE	CONDITIONS
className	FE & BE	non-empty; len 1–30; equals .java (case-insensitive); extracted .java class name == form name (case-insensitive)
classUTFile	FE & BE	present; not empty; readable; contains class declaration
classUTFileName	BE	cleaned path
robotTestsZip	FE & BE	present; not empty; extracts; contains robot-group folder
classUTDetails	BE	JSON parseable
difficulty	BE	one of {Beginner,Intermediate,Advanced}
description	FE & BE	if present len <= 150
category	FE & BE	if present size <= 3; each len <= 30
robot-group folders	BE	name ends with "Test"
robotType	BE	folder name without "Test"
levelFolders	BE	match \d{2,}Level
testFiles	BE	at least one .java per level

- **Reset del form HTML dopo un upload**

In seguito a un upload corretto di una classe, l'intero **form HTML** viene **resettato**, in modo da impedire un accidentale re-upload della stessa classe da parte dell'amministratore.

```

$('#successModal').modal('show');
//-----
// Se la classe è stata caricata correttamente, allora resetta il form.
const form = classInput?.form;
if (form) {
    form.reset();
}

```

- **Controller OpenAPI specification**

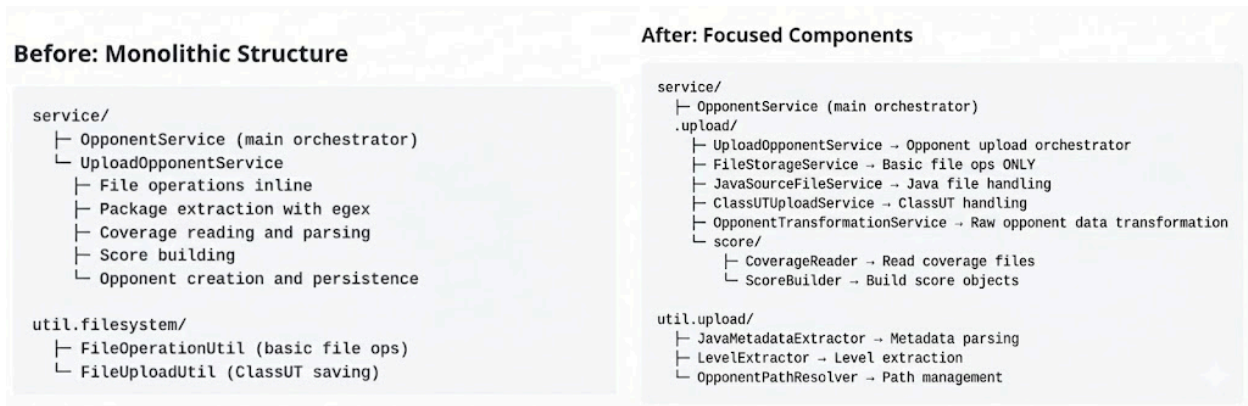
Aggiunte l'OpenAPI specification al controller di upload opponent

```

public class OpponentController {
    @PostMapping("")
    @ApiOperation(summary = "Upload class under test and robot tests",
        description = "Uploads a Java class file (ClassUT) along with its metadata and robot-generated test suites (EvoSuite/Randomop). " +
            "The endpoint validates the class file structure, verifies class name consistency, processes robot test archives, " +
            "and stores opponents with different difficulty levels. The robot tests zip must contain folders named 'EvoSuiteTest' " +
            "and/or 'RandomopTest' with subdirectories for each difficulty level (e.g., 01level, 02level) containing test files and coverage reports.")
    @ApiResponse(value = {
        @ApiResponse(
            responseCode = "200",
            description = "Successfully uploaded and processed class and robot tests. Returns metadata about the uploaded files including file names, URIs, and sizes.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = FileUploadResponse.class)
            )
        ),
        @ApiResponse(
            responseCode = "400",
            description = "Bad Request - Validation errors occurred. Possible causes: " +
                "empty or missing files, class name mismatch between form and source file, " +
                "invalid Java syntax, missing package declaration, invalid ZIP structure, " +
                "missing robot test directories (EvoSuiteTest/RandomopTest), or invalid test file format.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = ErrorResponseDTO.class)
            )
        ),
        @ApiResponse(
            responseCode = "409",
            description = "Conflict - A class with the same name already exists in the system.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = ErrorResponseDTO.class)
            )
        ),
        @ApiResponse(
            responseCode = "500",
            description = "Internal Server Error - Unexpected error occurred during file processing. " +
                "ZIP extraction, database operation, or robot test analysis.",
            content = @Content(
                mediaType = "application/json",
                schema = @Schema(implementation = ErrorResponseDTO.class)
            )
        )
    })
    public ResponseEntity<FileUploadResponse> uploadClassAndOpponents(

```


4. IMPLEMENTAZIONE E STRUTTURA DEL PROGETTO MODIFICATO



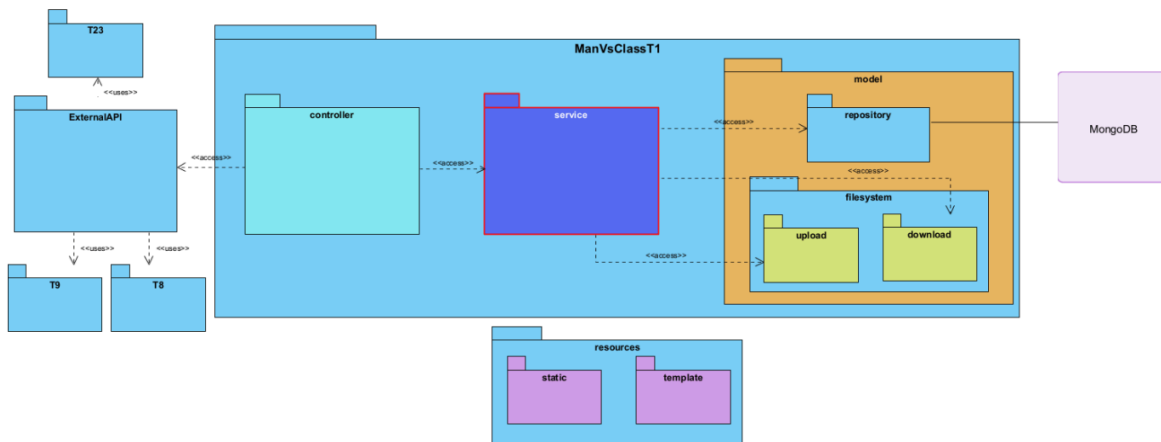
Package <i>com.groom.manvsclass.</i>	Classe	Responsabilità
service	OpponentService	orchestratore principale per l'upload della classe UT e degli opponent
service.upload	OpponentUploadService	orchestratore upload robot
service.upload	ClassUTUploadService	gestione upload class under tesst
service.upload	FileStorageService	operazioni sui file nel file system
service.upload	JavaSourceFileService	gestione file Java
service.upload	OpponentTransgormationService	trasformazione dei dati dei robot per convertirli alle strutture dati utilizzate per la persistenza
service.upload.score	CoverageReader	gestione ed estrazione dei dati dai file di coverage
service.upload.score	ScoreBuilder	generazione dei dati di score a partire dai dati dei file di coverage
util.upload	JavaMetadataExtractor	estrazione metadati dai file Java (es. package name)

util.upload	LevelExtractor	estrazione dei livelli dei robot di cui sono stati caricati i test
util.upload	OpponentPathResolver	path management (es. regex per estrazioni file zip)
controller	GlobalExceptionHandler	gestore di tutte le eccezioni nel micro servizio
config	FormValidation	validation dell'input del flusso di upload

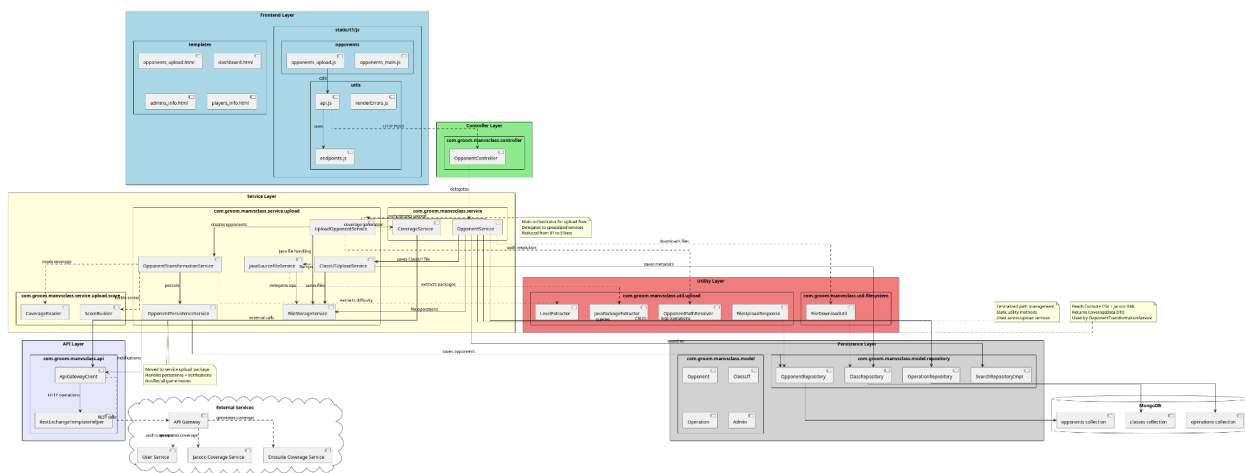
4.2 DIAGRAMMI AGGIORNATI

4.2.1 PACKAGE DIAGRAM (IBRIDO CON CLASS DIAGRAM)

PROGETTO INIZIALE:

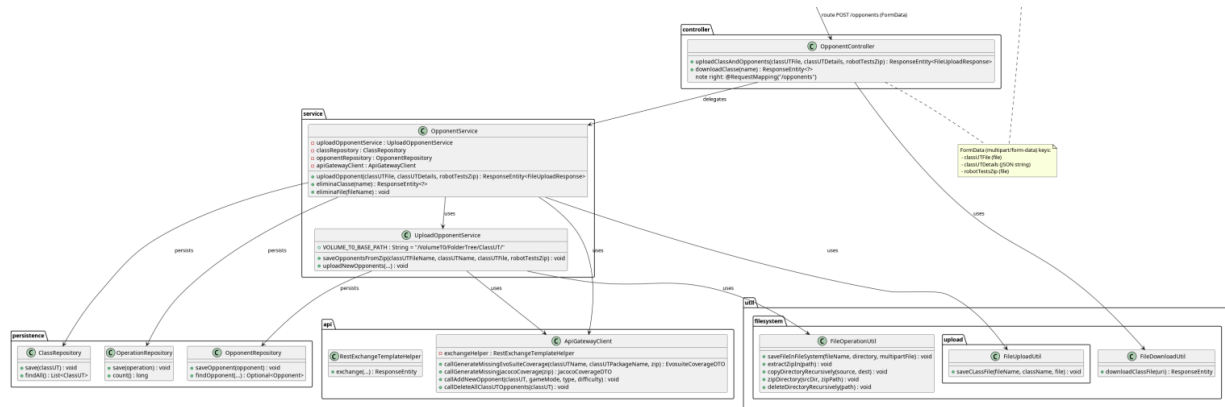


PROGETTO CONSEGNATO:

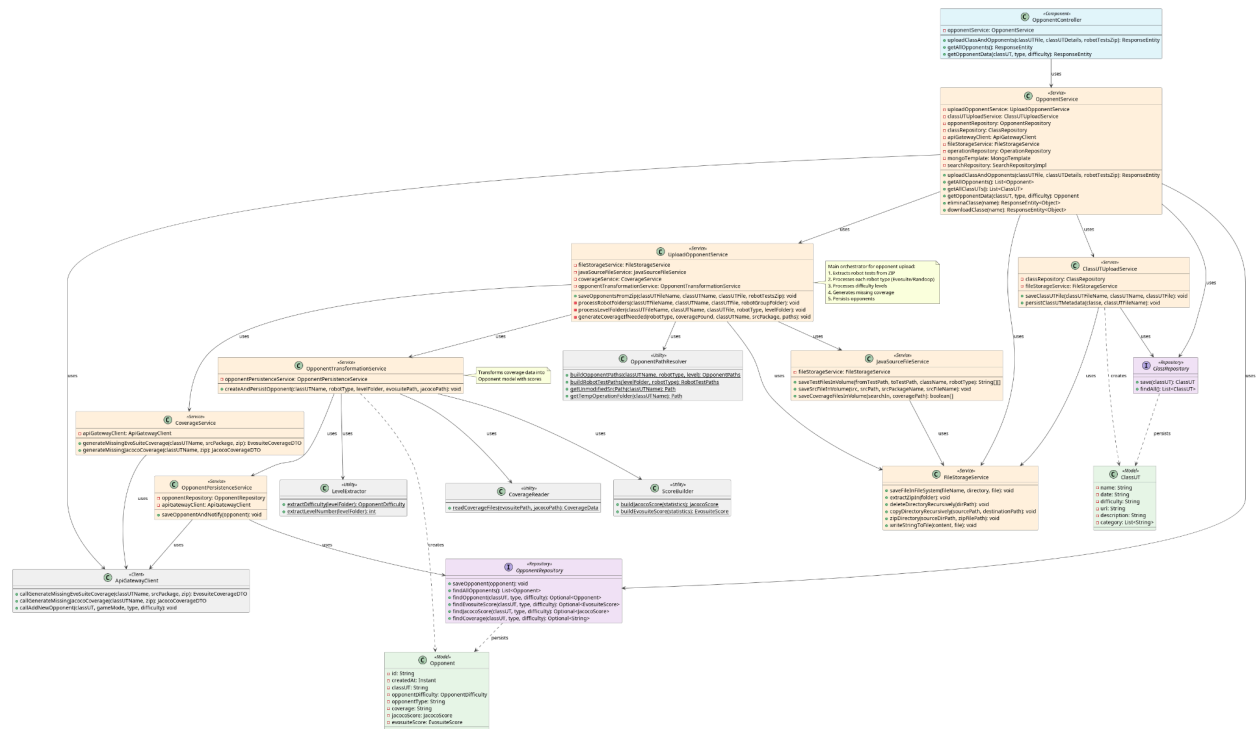


4.2.2 CLASS DIAGRAM

PROGETTO INIZIALE:

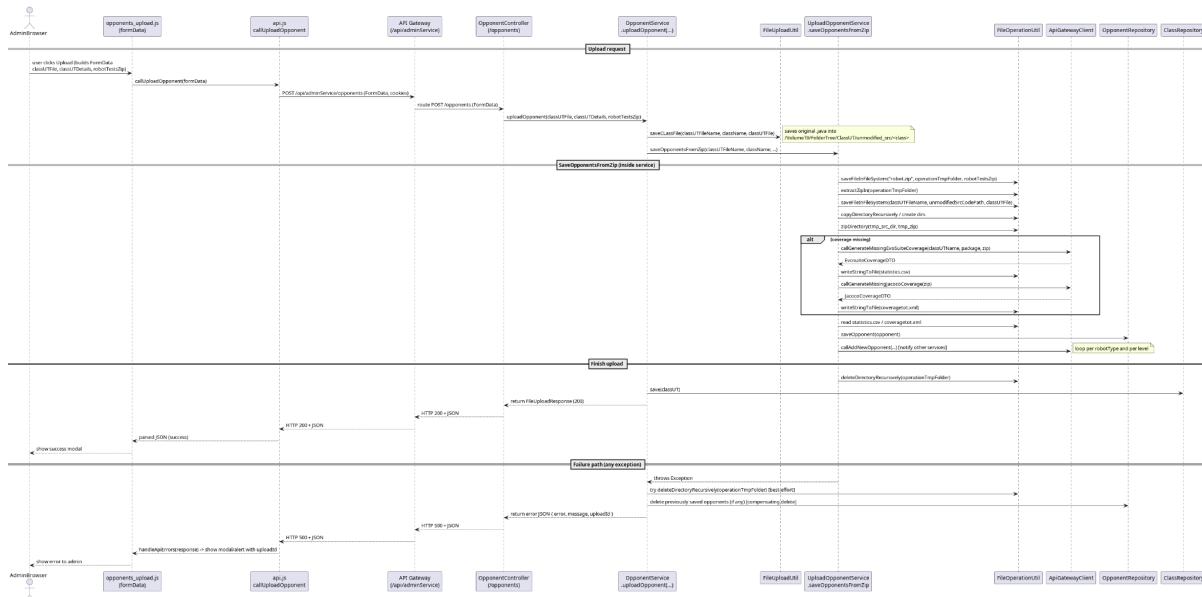


PROGETTO CONSEGNATO:

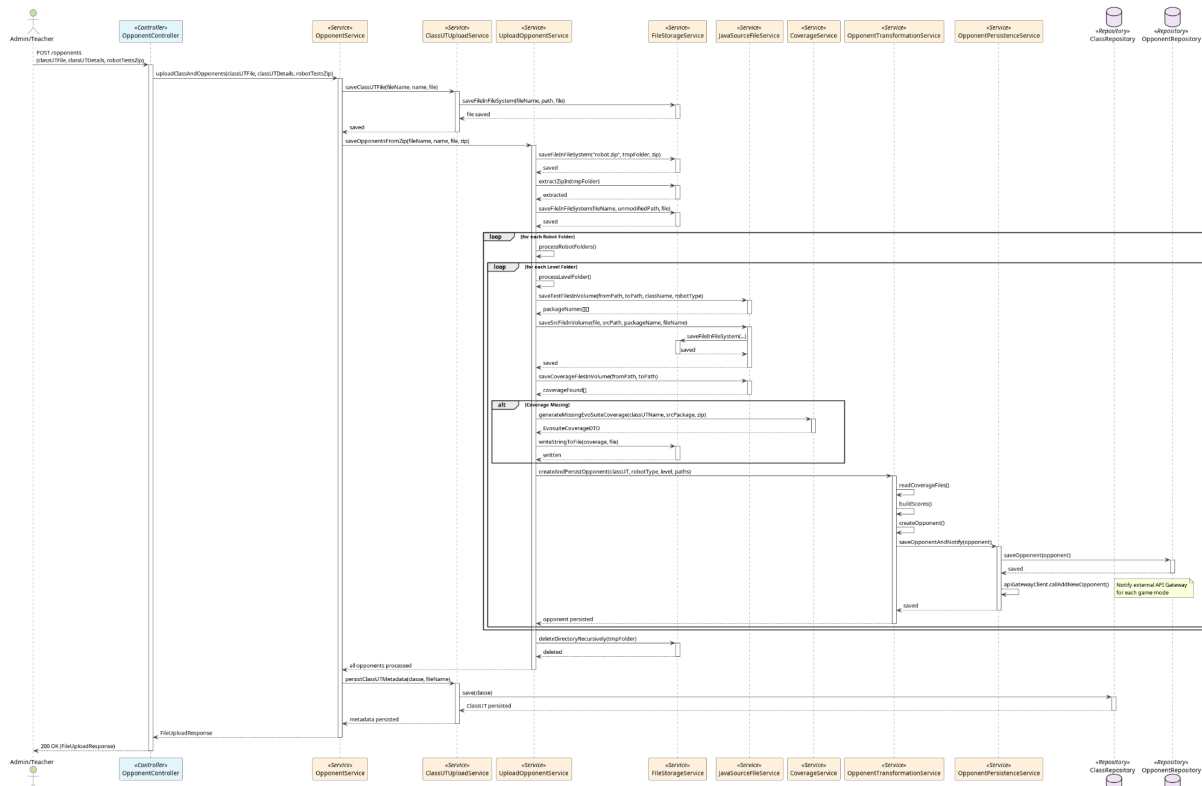


4.2.3 SEQUENCE DIAGRAM

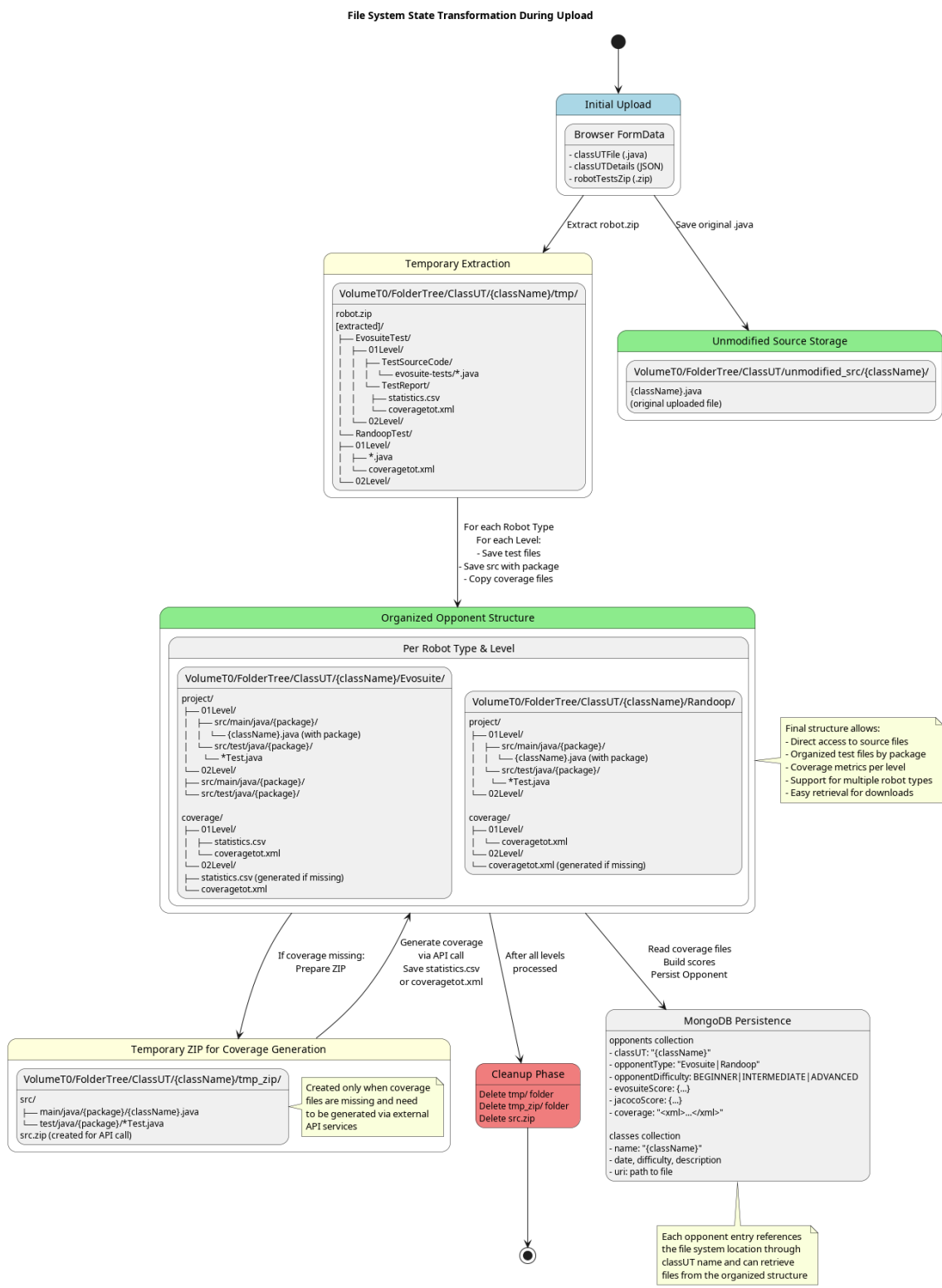
PROGETTO INIZIALE:



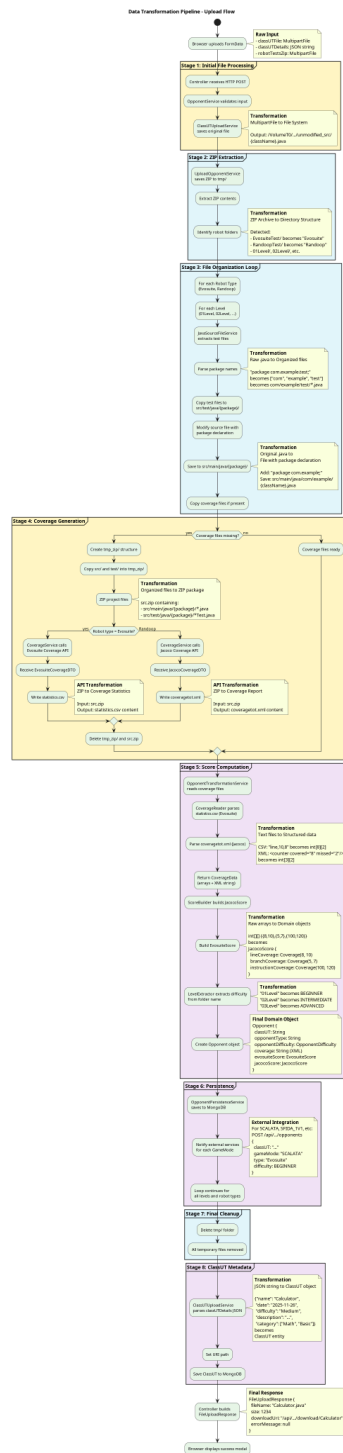
PROGETTO CONSEGNATO:



4.2.4 FILE PERSISTENCE DIAGRAM

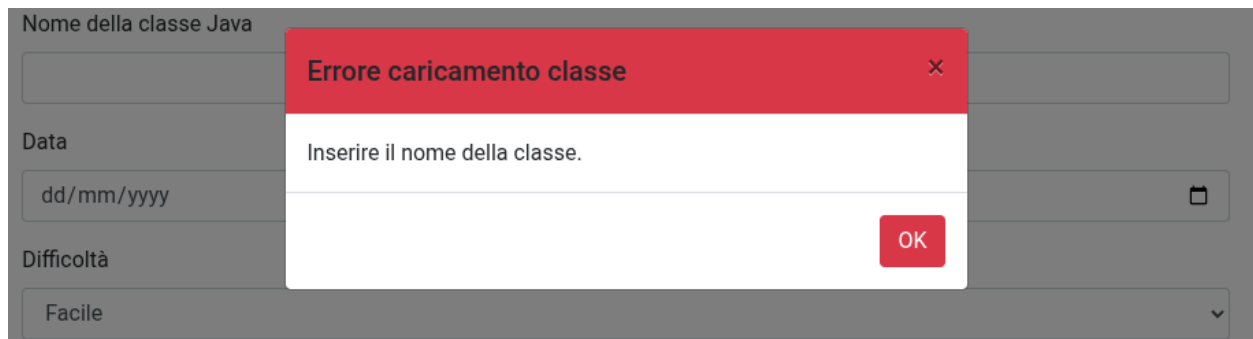


4.2.5 DATA TRANSFORMATION ACTIVITY DIAGRAM



5. DESCRIZIONE DEI TEST EFFETTUATI

- Test bug fix generazione file Evosuite con il file mancante nella cartella
- Test bug fix duplici chiamate a Evosuite e Jacoco
- Molteplici test con input non validi (class name non corrisponde, cartelle test mancanti, ..., esempi di input aggiunti nell'apposita cartella classiUT) per controllare il messaggio di risposta localizzato.
- Test con input valido e successiva partita con la nuova classeUT per verificare correttezza flusso di upload aggiornato



6. SVILUPPI FUTURI / RACCOMANDAZIONI

- Miglioramento log e risposte da **T7** e **T8**, poco parlanti e in alcune situazioni ambigue (es. http code 20x anche in caso di errore)
- Creazione Wiki